

Flexible Orchestration for AI & ML: Beyond Kubernetes Automation

Understand why AI and ML workloads outgrow a single Kubernetes cluster, and how a flexible workload orchestrator unifies services, batch jobs, and training runs under one scheduler.

For developers and DevOps engineers deploying AI/ML workloads who want to understand orchestration options beyond Kubernetes · 27 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

What matters

- ✓ A workload orchestrator automates deployment, scaling, health monitoring, and failure recovery from a declarative description, replacing manual server-by-server administration.
- ✓ Kubernetes was designed to keep long-running containerized services alive; that design assumption makes it awkward for ephemeral, run-once, or scheduled AI/ML workloads.
- ✓ AI/ML organizations commonly end up with four teams on four different tools — Kubernetes for web apps, Slurm for GPU training, Airflow for pipelines, and manual scripts or containers for ML services — which fragments troubleshooting expertise.
- ✓ A flexible orchestrator treats service jobs, batch jobs, and system jobs as first-class scheduler types, so one platform can run all of them instead of forcing every workload through the same primitive.
- ✓ A single declarative job spec bundles resource requirements, replica count, deployment strategy, and placement constraints, replacing the multiple Kubernetes manifests (Deployment, ConfigMap, Secret, storage) a single app usually needs.
- ✓ Consolidating onto one orchestrator turns fragmented ops and siloed per-tool knowledge into unified ops and shared knowledge, cutting a data scientist's turnaround from a multi-day ticket process to a few minutes of self-service.
- ✓ Because AI capabilities change unpredictably — transformer models and GPT-scale inference did not exist a few years ago — the orchestration layer needs to absorb new workload types without requiring the infrastructure itself to be rebuilt.

What it covers

01 The problem workload orchestration solves 0:00

Deploying the same application across many servers by hand means repeating the same manual login-and-install steps on every machine, and repeating them again for every fix.

02 Defining workload orchestration 1:25

A workload orchestrator runs many kinds of work — web apps, AI/ML jobs, batch jobs — across many machines while automating scheduling, placement, and health monitoring.

03 Why not just use Kubernetes? 2:54

A single Kubernetes application typically needs several separate manifests — Deployment, ConfigMap, Secret, storage — while a workload orchestrator can describe the same app in one declarative job file.

04 AI/ML workloads fragment across four tools 5:51

Organizations often split Kubernetes clusters by workload type, or cram everything into one cluster with heavy namespace and quota management — and different AI/ML teams end up on entirely different tools regardless.

05 **AI/ML workloads need flexible lifecycles** 7:16

Training jobs are ephemeral and run once, inference services must stay up continuously, and pipelines run on a schedule — three different lifecycles that don't map cleanly onto a platform built for one shape of workload.

06 **One scheduler, three first-class job types** 8:45

A flexible orchestrator treats batch jobs, services, and system jobs as equally first-class citizens of one scheduler, with each job's resource and placement needs declared directly in its spec.

07 **From fragmented ops to unified ops** 9:27

Consolidating onto one orchestrator replaces many tools plus siloed, tool-specific knowledge with one tool and shared knowledge across teams.

08 **Staying ready for what's next** 10:55

Because AI capabilities keep shifting unpredictably, a flexible orchestrator absorbs new workload types by adding a new job spec, not by rebuilding infrastructure.

Glossary

Workload orchestrator — Software that automates deploying, scaling, and healing applications across many servers from a declarative description, instead of requiring manual, per-server operation.

HCL (HashiCorp Configuration Language) — A human-readable declarative configuration language used to write job specs and other infrastructure definitions.

Batch job — A job type meant to run once (or on demand) to completion and then exit, such as a model-training run.

System job — A job type that runs one instance on every eligible node in a cluster, commonly used for agents or daemons.

Airflow — A workflow orchestration tool used to schedule and monitor multi-step data pipelines, typically modeled as directed acyclic graphs (DAGs).

Resource quota — A cluster-level limit on how much CPU, memory, or GPU capacity a namespace or team is allowed to consume.

Ephemeral workload — A job that runs briefly to complete one task and then terminates, rather than running indefinitely like a long-lived service.

Kubernetes — The most widely used container orchestrator, built primarily to keep long-running containerized services alive and self-healing.

Job spec — A declarative file describing an application's runtime requirements, replica count, resources, and deployment or placement strategy for an orchestrator to execute.

Service job — A job type meant to run continuously and be restarted automatically if it stops, such as an inference API.

Slurm — An HPC (high-performance computing) job scheduler commonly used in academic and research computing to queue and run GPU or CPU batch jobs.

Namespace (Kubernetes) — A way to partition a single cluster into isolated logical sections, often used to separate different teams or workloads sharing one cluster.

Placement constraint — A rule declared in a job spec that restricts where a workload is allowed to be scheduled, such as a specific datacenter or rack.

Explanations are AI-generated. Check anything you intend to rely on.